

CSC490 Final report - ArtGuard

Sohee Goo / 1008065479 Akshaya Deepak Ramachandran / 1008806810
Maryam Taj / 1008990362 Kashish Mittal / 1009115315

Contents

1	Introduction	2
2	Related Works	2
2.1	Academic	2
2.2	Industry	2
3	Technical Approach, Solution and Design	3
3.1	Solution Overview.	3
3.2	Vision Transformer Data.	4
3.3	Vision Transformer Pipeline.	4
3.4	RAG Knowledge Base Data	5
3.5	RAG Pipeline	5
4	Experiments, key metrics and results	7
4.1	Experimental Setup	7
4.2	Motivation for Ablations	7
4.3	Baseline: No Augmentation	7
4.4	Rotation Only	8
4.5	Gaussian Blur Only	8
4.6	Rotation + Gaussian Blur	8
4.7	Comparative Analysis	8
4.8	Final Model Selection	9
4.9	Discussion and Limitations	9
4.10	Summary	9
5	Future Work and Next Steps	9
5.1	RAG & Explanation Quality	9
5.2	Security	9
5.3	Scaling to Multiple Artists	9
5.4	Training & Inference Efficiency	9
6	Team Reflection and Connection to Course Material	10
6.1	Major Learning Curves	10
6.2	Connection to Course Material	10
7	Sources	11

1 Introduction

Art forgery represents a long-standing and significant problem in the global art market. With the high value associated with artworks, particularly those attributed to well-known artists, there is strong incentive for the production and circulation of forged works. Industry experts estimate that a substantial portion of artworks in circulation may in fact be misattributed or forged, contributing to widespread uncertainty within the art market [1]. Furthermore, authenticating a single artwork often requires a combination of expert domain knowledge, provenance records, and scientific analysis, making the process both expensive and time consuming.

Professional authentication services frequently rely on a combination of art historical expertise, material analysis, and imaging techniques to determine authenticity. These services can cost thousands of dollars per artwork, creating barriers for smaller galleries, collectors, and institutions [1]. As a result, much of the art market operates under conditions of incomplete information, where verifying authenticity can be prohibitively costly and slow.

Recent advances in machine learning, particularly in computer vision, have introduced new possibilities for automating aspects of the authentication process. Deep learning models can identify stylistic patterns in artworks by analyzing features such as brushstroke texture, color distributions, and compositional structures [2]. Models such as convolutional neural networks (CNNs) and vision transformers (ViTs) have demonstrated promising performance when trained on datasets of artworks [3]. Despite these advances, most existing approaches focus on generating a binary classification of authentic versus forged [4]. These systems often require substantial computational resources or rely on expensive, proprietary datasets, limiting their accessibility to users outside elite galleries or high-budget institutions.

This project explores an AI-assisted approach to art authentication that aims to make the process faster, more cost-effective, and widely accessible. By combining deep learning models with a retrieval-augmented generation (RAG) pipeline, the system leverages structured metadata and visual features to support automated authentication without relying on expensive imaging equipment or lengthy expert analysis. While the system can provide contextual information about stylistic patterns and artwork metadata, the primary goal is to deliver a practical, scalable tool that lowers the barriers for small galleries, collectors, and independent researchers seeking to assess artwork authenticity in a timely and affordable manner.

2 Related Works

2.1 Academic

Recent research has explored the application of deep learning techniques to the problem of art authentication. Early approaches primarily relied on CNNs to extract hierarchical visual features from artworks and classify them according to artist or authenticity. For example, residual neural network architectures have been used to analyze datasets such as the Rijksmuseum collection, demonstrating that deep learning models can capture stylistic patterns characteristic of specific artists [5].

More recent work has investigated transformer-based architectures for art authentication tasks. Vision Transformers (ViTs) and hybrid models combining convolutional layers with transformer blocks have shown promising results in capturing long-range dependencies within images. Studies comparing CNNs with transformer-based models have found that architectures such as Swin Transformers can outperform traditional CNN models in certain authentication settings, particularly when distinguishing between closely related artistic styles [3].

Patch-based analysis has also emerged as a common strategy in art forgery detection. Rather than analyzing entire paintings as a single image, many approaches divide artworks into smaller patches in order to capture fine-grained stylistic features such as brushstroke patterns or texture variations [2]. These patches can then be used to train models that identify subtle differences between authentic works and forgeries.

The main challenge frequently discussed in the literature is the scarcity of labeled forgery data. Since authentic artworks are far more widely documented than confirmed forgeries, datasets often suffer from significant class imbalance [4]. Some researchers have addressed this issue by generating synthetic training data using generative models. Studies have shown that incorporating AI-generated forgeries into training datasets can improve model performance by helping models learn clearer stylistic boundaries between authentic works and imitations.

Beyond classification accuracy, recent research has also begun to explore interpretability and explainability in image forgery detection. Systems such as So-Fake integrate detection and localization capabilities, allowing models to identify regions within images that contribute to their predictions [6]. While these systems primarily focus on digital image manipulation in social media contexts, they highlight the importance of interpretable outputs for real-world applications.

2.2 Industry

In parallel with academic research, several companies have developed commercial solutions for art authentication using machine learning and advanced imaging technologies. These services are typically positioned as high-end tools used by major auction houses, banks, and galleries.

One prominent example is Art Recognition AG, a Swiss company that applies deep learning models to the authentication of artworks [7]. Their system reportedly uses hybrid neural architectures combining convolutional networks with transformer-

based components. While the company has demonstrated strong performance in detecting stylistic inconsistencies, the reports generated by the system typically provide only binary authentication decisions with limited explanation of the underlying reasoning.

Other organizations integrate machine learning with traditional forensic and art historical analysis. For example, Hephaestus Analytical combines AI-based brushstroke analysis with chemical material testing, provenance research, and expert evaluation [8]. While this hybrid approach can provide stronger authentication guarantees, it often requires expensive imaging equipment and extensive manual analysis, making the service inaccessible to much of the art market.

Some companies have also explored novel approaches to addressing the scarcity of forgery datasets. Norval AI, for instance, generates training data by using robotic systems to replicate an artist’s drawing style, producing high-quality synthetic forgeries that can be used to train detection models [9]. While this method can generate realistic training data, it requires direct collaboration with living artists and is therefore less applicable to historical artists.

Taken together, these industry solutions demonstrate the growing interest in AI-assisted art authentication. However, they also highlight several limitations, including high costs, limited interpretability, and long wait times. These limitations suggest an opportunity for more scalable and transparent tools that can support a broader range of users within the art ecosystem.

3 Technical Approach, Solution and Design

3.1 Solution Overview.

We implemented an end-to-end art forgery detection system that enables users to upload artwork images and receive both a quantitative authenticity score and a natural-language explanation grounded in art-historical knowledge. ArtGuard addresses the absence of accessible, automated forgery detection tooling by combining deep learning inference with retrieval-augmented explainability atop production-grade cloud infrastructure.

At its core, the user workflow is straightforward: a user uploads an artwork image with artist and artwork metadata, and the system returns a forgery prediction with a confidence score and a plain-language explanation citing relevant art-historical context. Users can review past analyses, view patch-level heatmaps showing which regions the model flagged as suspicious, and export PDF reports.

The system is organized around three core components:

1. **Patch-level vision model inference.** Following Schaerf et al. [3], uploaded images are decomposed into a resolution-dependent grid of 224×224 patches and classified individually by a fine-tuned Swin Vision Transformer (PyTorch) running on Modal serverless GPUs. Patch-level probabilities are aggregated into a single painting-level authenticity score via mean pooling.
2. **RAG explainability layer.** Each inference result is paired with a natural-language explanation generated by Claude 4.5 Sonnet via Amazon Bedrock, grounded in a curated knowledge base of art-historical documents from the Metropolitan Museum and Wikidata, indexed in an OpenSearch Serverless vector store with Titan Embed v2 embeddings.
3. **Full-stack application and cloud infrastructure.** A Python FastAPI backend on ECS Fargate serves a React 18 / TypeScript frontend delivered via CloudFront. User data is persisted across six DynamoDB tables and four S3 buckets. Authentication uses JWT (HS256) with bcrypt-hashed passwords.

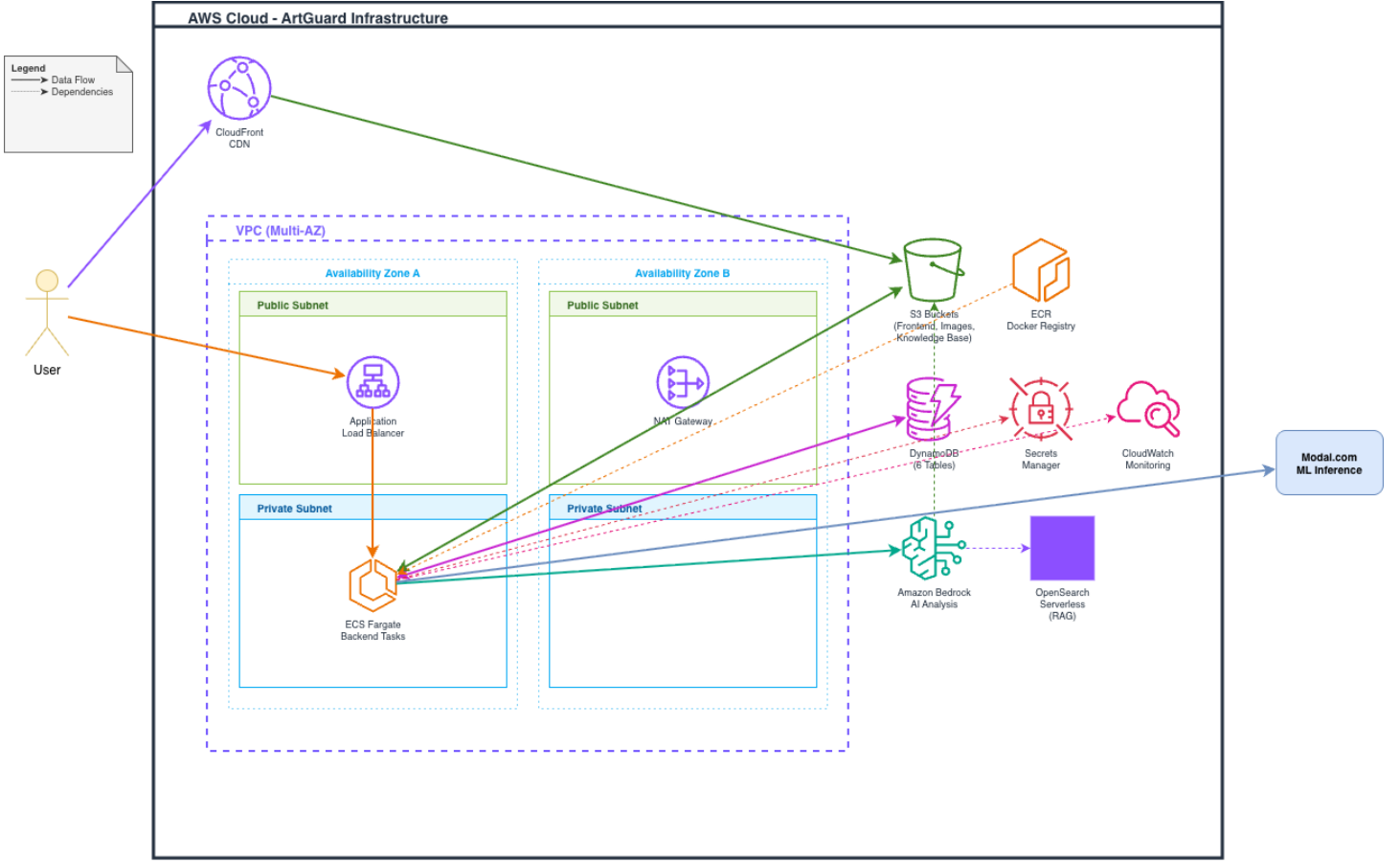


Figure 3.1 presents the high-level system architecture, illustrating the data flow from the CloudFront CDN through the Application Load Balancer to ECS Fargate, which orchestrates calls to Modal for ML inference, Bedrock for RAG explanation, DynamoDB for persistence, and S3 for object storage.

3.2 Vision Transformer Data.

We train the model using the dataset from “*Synthetic images aid the recognition of human-made art forgeries*”, which contains Van Gogh-related images from four categories: authentic (124), proxies (212), imitations (9), and diffusion (30), for a total of 375 samples. Since our objective is binary forgery detection, we map labels as follows: class 1 corresponds to *authentic* images, while class 0 corresponds to *everything else* (proxies, imitations, and diffusion).

Bronze/Silver/Gold Partitioning. For reproducible dataset organization, we partition data into three stages:

- **Bronze:** raw input images as provided by the dataset.
- **Silver:** derived 224×224 patch inputs produced by the patch extraction pipeline.
- **Gold:** the final image-level prediction outputs produced by the model (binary authenticity decision and corresponding score).

Data Partitioning. At the split level, we follow the paper-defined train/val/test partition for preventing data leakage and enabling direct comparison with reported results. Labels are applied at the image level using the binary convention above, and the patch dataset inherits the parent image’s binary label. Stratification is used during split assignment to maintain consistent category proportions across all partitions. During optimization, the model is trained on patches from the training split and evaluated using the validation split; the test split is reserved for final reporting.

3.3 Vision Transformer Pipeline.

Given an input image, we first split it into resolution-dependent grids. Let the minimum image side be s . We construct an $N \times N$ grid as follows: if $s > 1024$ we use $N = 4$ (16 cells); otherwise if $s > 512$ we use $N = 2$ (4 cells); and for smaller images we also use $N = 2$.

Each grid cell produces two local views for fine-grained classification:

- **Center crop patch:** extract the central 224×224 region from the cell.
- **Downsample patch:** resize the full cell to 224×224 using bicubic resizing.

All patches are then converted to tensors and normalized using ImageNet mean/std statistics. During training, we apply standard augmentation (random horizontal flip), while validation/test use deterministic preprocessing (no augmentation).

The resulting set of 224×224 patch tensors is fed into a **Swin-Base** backbone. The Swin model is modified with a classification head producing a single *logit* per patch, z_i , which is converted to an authenticity probability via the sigmoid function:

$$p_i = \sigma(z_i).$$

Image-Level Aggregation and Output. Because each image yields multiple patches, we aggregate patch probabilities into a single painting-level authenticity score by averaging:

$$\text{mean_prob} = \frac{1}{N} \sum_{i=1}^N p_i.$$

The final binary prediction is then obtained by thresholding the aggregated score:

$$\hat{y} = \begin{cases} 1 & \text{if mean_prob} > 0.5 \\ 0 & \text{otherwise.} \end{cases}$$

Training Objective (Weighted BCE with Logits). Training uses binary cross-entropy computed directly on logits with `BCEWithLogitsLoss`. For each patch sample, we compute a per-sample loss $\ell(z_i, y_i)$ and apply a sample weight w_i to up-weight inauthentic examples. Authentic patches ($y_i = 1$) use $w_i = 1.0$, while inauthentic patches ($y_i = 0$; proxies/imitations/diffusion) are up-weighted by `imitation_weight` (default 10.0). The final batch loss is the weighted mean:

$$\mathcal{L} = \text{mean}(w_i \cdot \ell(z_i, y_i)).$$

Validation loss is computed with the same logits-based BCE objective, using an unweighted mean across samples.

3.4 RAG Knowledge Base Data

To construct the knowledge base used in our retrieval-augmented generation (RAG) pipeline, we relied on two primary data sources containing structured metadata about artists and artworks.

The first and primary source was Wikimedia Commons [10]. Using SPARQL queries, we extracted structured information related to artists and artworks included in our image dataset, such as Vincent van Gogh and Otto Wacker. This process allowed us to retrieve relevant metadata including artist information, artwork descriptions, and historical context. The retrieved data was cleaned prior to uploading to an S3 bucket, retaining only metadata relevant to our task, such as artist inspirations, movements, and artwork materials. By aligning the knowledge base with the artists represented in our dataset, we ensured that the RAG pipeline could retrieve relevant contextual information during both training and evaluation.

In addition to Wikimedia Commons, we incorporated metadata from the Metropolitan Museum of Art (MET) Open Access dataset [11]. This dataset contains metadata for over 500,000 artworks spanning a wide range of artists, time periods, and artistic styles. Incorporating this dataset allowed us to expand the scope of the knowledge base and improve the system’s ability to generalize beyond the specific artists present in our training images. As a result, the RAG system can retrieve contextual information from a broader corpus of art historical data when generating explanations.

3.5 RAG Pipeline

The RAG pipeline begins with a query to the knowledge base. This query consists of the artist name followed by the artwork name, as provided by the user at the start of the inference pipeline. The query is used to retrieve the top seven most relevant chunks from the knowledge base. We selected seven chunks because our chunking strategy often split related information across multiple segments. Retrieving seven chunks ensured that sufficient metadata was available for the generation stage. This retrieval step ensures that the generated explanation is grounded in factual metadata about the artist and artwork rather than relying solely on the language model’s internal knowledge.

To perform retrieval, we used the Titan Embeddings v2.0 model to generate vector representations of the query and the stored knowledge base chunks. This model was selected because it performs well for textual embeddings and offered a favorable cost-performance trade-off given that our knowledge base contains only textual information.

The retrieved metadata is then passed to the generation stage of the RAG pipeline. For this stage, we used the Claude Sonnet 4.5 model as the generation model. This model was selected based on its strong performance while, once again,

maintaining a reasonable cost-quality trade-off. We initially experimented with the Claude Haiku 3 model, however, during evaluation we observed that the model frequently produced hallucinated or incomplete explanations. As a result, we determined that Claude Sonnet 4.5 was better suited for generating reliable and coherent explanations for this particular task.

In addition to the retrieved metadata, the language model is provided with several inputs from the classification pipeline. These include the predicted classification label (authentic or inauthentic), a confidence score indicating the model’s certainty in its prediction, the top eight image patches that contributed most strongly to the prediction, and the original image data. The confidence score is calculated using the following formula:

$$confidence = \frac{|mean_prob_of_scores - 0.5|}{0.5} \times 100$$

We provide the top eight patches because each image is divided into at least eight patches during preprocessing, ensuring that a consistent set of influential regions is available for explanation generation.

Finally, the language model is prompted to generate an explanation for the predicted label using the provided inputs. The prompt is structured using XML-style tags to define the model’s role and clarify the task it must perform. This prompt design follows best practices outlined in Anthropic’s prompt engineering documentation to encourage structured, grounded responses [12].

The generated explanation is then returned to the user as part of the overall inference pipeline.

Backend API. The backend API exposes 17 REST endpoints organized across six route modules. The application follows a layered architecture: route handlers are thin orchestrators that validate requests via Pydantic models and delegate to a service layer containing all business logic and AWS interactions, while data access uses LRU-cached boto3 client factories and a pagination abstraction over DynamoDB’s cursor mechanism. A custom ASGI middleware wraps every request with a correlation ID (from the `X-Request-ID` header or auto-generated), structured JSON logging of method, path, status, and duration, and async-safe `ContextVar` propagation of `request_id` and `user_id` through the call stack. The API is organized into four functional groups: authentication (5 endpoints: signup, login, profile retrieval, profile update, password change with JWT tokens and bcrypt hashing), inference (6 endpoints: image submission via multipart upload, paginated history with cursor-based pagination, single record retrieval, statistics, and individual or bulk deletion), training and evaluation (2 endpoints: asynchronous Modal job spawning with DynamoDB run tracking), and RAG/data processing (2 endpoints: free-text knowledge base queries and ECS Fargate task launching for image preprocessing).

Infrastructure and Design Decisions. ArtGuard’s infrastructure is shaped by one core observation: the system has two fundamentally different compute profiles—a web API that mostly serves lightweight requests in milliseconds, and GPU-intensive ML workloads that run infrequently but need dedicated VRAM. We chose Modal Labs’ serverless GPU platform over self-managed EC2 GPU instances because art forgery detection is inherently bursty: a user uploads an image, waits for a result, and may not return for hours, so a dedicated GPU would be wasteful. Modal’s pay-per-second billing means we pay only during actual compute, and the cold-start trade-off is absorbed into the 10–30 second inference pipeline that users already expect. The backend API itself runs on ECS Fargate rather than Lambda because inference requests are long-running and benefit from persistent cached connections across requests. We chose DynamoDB over RDS after analyzing our actual query patterns: every access is a single-key lookup or a one-to-many GSI scan—there are no heavy joins, no aggregations, and no multi-entity transactions, so the relational model would add operational overhead without benefit. DynamoDB’s on-demand billing matches our spiky traffic. Training is deliberately offline and manually triggered because the forgery dataset changes rarely (new examples require expert labeling), training takes hours on A10G GPUs, and model quality requires human review of per-sublabel metrics. Because patch batches for a single image are small (4–32 patches of 224×224), the T4’s VRAM is sufficient and the additional throughput of the A10G does not justify its 87% cost premium. Training, which processes thousands of patches per epoch in larger batches, benefits from the A10G’s higher memory bandwidth and is run on A10G accordingly. For RAG, we chose Bedrock’s managed Knowledge Base over building retrieval ourselves because it handles embedding, vector search, and generation in a single API call; the trade-off is less control over retrieval tuning, but the RAG stage is designed for graceful degradation so inference results are never blocked by explanation failures. All approximately 60 AWS resources are defined as Terraform infrastructure-as-code, enabling the full stack to be provisioned from scratch in under 30 minutes, critical for disaster recovery testing. Monitoring uses CloudWatch structured JSON logs with custom embedded metrics for inference and RAG latency, and 11 GitHub Actions workflows automate testing, deployment, and infrastructure management on every push.

Security, Networking, and Reliability. Because ArtGuard accepts user-uploaded images and free-text metadata from the public internet, input validation is a first-class concern. Filenames are sanitized against path-traversal sequences, null bytes, and leading dots to prevent malicious uploads from escaping the intended S3 prefix, and the test suite explicitly includes XSS and SQL injection strings in metadata fields to verify they are rejected or escaped safely. All data at rest is encrypted (AES-256 for S3, AWS-managed KMS for DynamoDB and Secrets Manager), and all data in transit uses TLS 1.2+, because the system handles user credentials and artwork images that may have commercial sensitivity. ECS tasks

run in private subnets with no public IP addresses, receiving traffic exclusively from the ALB via security groups scoped to port 8000—this ensures that even if an attacker discovers the internal service address, they cannot reach it without going through the load balancer. Five VPC Endpoints provide private connectivity to S3, ECR, CloudWatch, and Secrets Manager, so internal AWS API calls never traverse the public internet. IAM roles follow least-privilege: the ECS Task Role is scoped to specific table and bucket ARNs rather than wildcard permissions, meaning a compromised container cannot access resources outside the application’s scope. System reliability is validated through three tiers of testing: 626 backend tests using Moto (which creates real in-memory DynamoDB tables with the same schemas and GSIs as production, catching mismatches that return-value stubs would miss), 247 frontend unit tests, and 66 Playwright E2E tests—all at 100% line coverage, enforced in CI on every push. The frontend includes a built-in mock mode (`hasApiBackend()`) that runs the full application against `localStorage` when no backend URL is configured; this was a deliberate architectural choice that enabled parallel frontend/backend development across team members and deterministic E2E testing without standing up AWS infrastructure.

Disaster Recovery and Deployment. Disaster recovery uses Terraform `state rm` to detach data-bearing resources (six DynamoDB tables, two S3 buckets) before destroying infrastructure, then `import` to re-adopt them during recovery—the entire destroy/verify/recover cycle is executable via a single script with zero data loss. We chose this state-manipulation approach because our AWS account is restricted to a single region, ruling out cross-region replication which would be the standard production strategy. In a real production environment, we would use S3 Cross-Region Replication and DynamoDB Global Tables for automatic multi-region failover; our approach achieves the same data-preservation guarantee within single-region constraints. DynamoDB Point-in-Time Recovery is already enabled in production for protection against accidental data corruption.

4 Experiments, key metrics and results

4.1 Experimental Setup

We evaluate a Swin-Base vision transformer on a dataset of 375 paintings, consisting of 124 authentic Van Gogh works, 212 proxy paintings, 9 imitations, and 30 diffusion-generated images. Due to the limited dataset size and class imbalance, we adopt a patch-based classification approach, where each painting is divided into 224×224 patches. Patch-level predictions are aggregated via mean pooling to produce painting-level predictions.

We report results at both the patch level and painting level, with emphasis on painting-level performance as it reflects the real-world task. Given the importance of correctly identifying authentic works, we focus on accuracy, precision, recall, and F1-score, with particular attention to recall for authentic paintings.

4.2 Motivation for Ablations

We investigate two data augmentation strategies motivated by prior work in art authentication and image-based classification.

Gaussian blur. Prior work in art forgery detection highlights the importance of reducing high-frequency noise and superficial artifacts introduced during image acquisition. In particular, *Art Forgery Detection using Kolmogorov Arnold and Convolutional Neural Networks* demonstrates that applying Gaussian blur can mitigate digital noise and improve model robustness by encouraging focus on coarse stylistic features rather than fine-grained pixel artifacts. [13]

Rotation. Data augmentation through geometric transformations such as rotation has been widely used to prevent overfitting in image classification tasks. *Contemporary Art Authentication with Large-Scale Classification* applies random rotations and other transformations during training to improve generalization across diverse image orientations. [14]

Based on these motivations, we conduct an ablation study with four configurations: (1) no augmentation (baseline), (2) rotation only, (3) Gaussian blur only, and (4) rotation + Gaussian blur.

4.3 Baseline: No Augmentation

We first train the Swin-Base model without any augmentation to reproduce the reference setting. The best checkpoint is obtained at epoch 2, achieving a validation accuracy of 0.877 and validation loss of 0.398.

At the painting level, the model achieves:

- Accuracy: 0.857
- Precision: 0.833
- Recall: 0.714
- F1-score: 0.769

The model converges rapidly but exhibits early overfitting, with validation performance peaking after two epochs. While proxy and diffusion images are classified accurately, the model misclassifies 6 out of 21 authentic paintings, indicating low recall and a tendency toward false negatives on genuine works.

4.4 Rotation Only

We next evaluate rotation augmentation in isolation. The best performance occurs at epoch 1, with validation accuracy of 0.852.

At the painting level:

- Accuracy: 0.857
- Recall: 0.714
- F1-score: 0.769

Performance is nearly identical to the baseline, while overfitting occurs even earlier. This suggests that geometric invariance is not a dominant factor in distinguishing authentic from non-authentic paintings in this dataset.

4.5 Gaussian Blur Only

We then evaluate Gaussian blur augmentation. The best checkpoint is obtained at epoch 5, with validation accuracy of 0.862.

At the painting level:

- Accuracy: 0.889
- Precision: 0.850
- Recall: 0.810
- F1-score: 0.829

Gaussian blur produces a clear improvement across all metrics. In particular, recall increases from 0.714 to 0.810, reducing the number of misclassified authentic paintings from 6 to 4. Additionally, training remains stable for longer, indicating reduced overfitting.

4.6 Rotation + Gaussian Blur

Finally, we evaluate the combination of both augmentations. The best checkpoint is obtained at epoch 7.

At the painting level:

- Accuracy: 0.889
- Precision: 0.850
- Recall: 0.810
- F1-score: 0.829

Performance is identical to the blur-only setting, although the model generalizes slightly longer before overfitting.

4.7 Comparative Analysis

Model Variant	Accuracy	Recall	F1-score	Best Epoch
Baseline	0.857	0.714	0.769	2
Rotation only	0.857	0.714	0.769	1
Blur only	0.889	0.810	0.829	5
Rotation + Blur	0.889	0.810	0.829	7

Table 1: Painting-level performance across augmentation strategies

Gaussian blur is the primary contributor to performance gains, improving both recall and overall robustness. Rotation alone provides negligible benefit, but when combined with blur, it slightly delays overfitting without improving final performance.

4.8 Final Model Selection

Based on the ablation study, we select **Gaussian blur augmentation** as our final configuration. While combining rotation with blur yields similar performance, blur alone achieves the same accuracy with earlier convergence and reduced training cost. More importantly, blur significantly improves recall on authentic paintings, which is the most critical metric for art authentication. Reducing false negatives ensures that genuine works are less likely to be incorrectly rejected.

4.9 Discussion and Limitations

Despite improvements from augmentation, our best model achieves a painting-level accuracy of 0.889 and F1-score of 0.829, which remains below the reference paper’s reported performance (0.866 accuracy, 0.960 F1).

Several factors likely contribute to this gap:

- Limited dataset size (375 paintings)
- Strong class imbalance, particularly for imitation samples
- Noise introduced by patch-level aggregation

Additionally, imitation detection remains weak across all configurations, indicating that human imitations are the most challenging class due to their stylistic similarity to authentic works.

4.10 Summary

Overall, our results demonstrate that augmentation is critical for improving generalization in art authentication. Gaussian blur is particularly effective, as it encourages the model to focus on coarse stylistic patterns rather than fine-grained texture noise. In contrast, rotation provides minimal benefit, suggesting that stylistic invariance is more important than geometric invariance in this domain.

5 Future Work and Next Steps

5.1 RAG & Explanation Quality

The authenticity explanation is a reasoning-heavy task requiring patch scores, retrieved art context, and artist metadata. Structuring the Bedrock prompt with an explicit **chain-of-thought** strategy—analyze patches, cross-reference retrieved documents, then synthesize—would improve consistency and trustworthiness. **Constrained decoding** can be implemented via a tools definition with a JSON Schema and `"tool_choice": {"type": "tool", "name": "provide_explanation"}` in the Bedrock `invoke_model` call, forcing Claude to return structured JSON (verdict, patch analyses, sources) and preventing missing citations or formatting drift.

5.2 Security

ArtGuard uses hierarchical prompting and grounding constraints, but user inputs (`artist_name`, `artwork_name`) are not escaped. Hardening should include: (1) escaping XML control characters to preserve the `<analysis_input>` structure, (2) detecting injection patterns (e.g., “ignore previous instructions”), and (3) sanitizing retrieved KB chunks to prevent indirect prompt injection from poisoned data.

5.3 Scaling to Multiple Artists

The binary classifier can be extended to multi-class (original, forgery, imitation, proxy) using existing sublabels. For under-represented artists, **LoRA adapters** can capture artist-specific features without retraining the full model. Serving can be optimized by loading a single base model (`best.pt`) in GPU memory and swapping lightweight (~2MB) adapters per request instead of loading separate 100MB+ models, with routing based on artist name and fallback to the base model.

5.4 Training & Inference Efficiency

Mixed precision (FP16) training on A10G GPUs would roughly double speed and halve memory usage with negligible accuracy loss. Additional improvements include `torch.compile` (10–30% speedup), local patch caching to remove S3 bottlenecks, and stronger augmentation (rotation, color jitter). At inference, switching from FP32 to FP16 (`torch.cuda.amp.autocast()`) or INT8 quantization on T4 GPUs would significantly increase throughput using dedicated hardware support, with minimal accuracy impact and no changes to training or checkpoints.

6 Team Reflection and Connection to Course Material

6.1 Major Learning Curves

A key lesson was that production ML systems require strong infrastructure discipline beyond model performance. One of the primary challenges we encountered was coordinating multiple asynchronous components—ML inference, RAG retrieval, and API responses—while maintaining consistent latency and system reliability. Each stage of the pipeline had different execution characteristics (GPU-bound inference, network-bound retrieval, and CPU-bound API handling), which made it difficult to reason about end-to-end performance and failure modes. For example, delays or failures in the RAG pipeline could block user responses even when the classifier succeeded, requiring us to design the system to degrade gracefully.

To address these challenges, we adopted a modular architecture with clear separation of concerns and explicit service boundaries. Infrastructure-as-code via Terraform (with S3 remote state and DynamoDB locking) was critical in managing complex dependencies across ECS, ALB, ECR, S3, CloudFront, and Bedrock, while ensuring reproducibility and safe iteration. We also used GitHub Actions to automate Terraform validation and deployment, container builds to ECR, and frontend deployment to S3/CloudFront, which reduced configuration drift and enabled rapid iteration across the full stack.

A second major learning curve was designing for cost-efficiency without sacrificing performance. Because ML inference is GPU-intensive but occurs in bursts, we separated responsibilities across services: FastAPI on ECS for lightweight API handling, Modal serverless GPUs (T4) for Swin-based patch inference, and Bedrock for retrieval and explanation. This architecture improved scalability and reduced idle compute cost, but introduced new challenges around authentication, secret management, and cross-service observability. In particular, coordinating credentials and ensuring secure communication between ECS, Modal, and Bedrock required careful IAM scoping and secret storage.

Finally, we found that maintaining experimentation velocity in a distributed system required strong tooling support. Integration of Weights & Biases (W&B) enabled systematic tracking of training metrics and ablations, while Modal simplified remote GPU experimentation without manual infrastructure management. Together, these tools allowed us to iterate quickly on model improvements while maintaining reproducibility across runs.

Overall, this project highlighted that building reliable ML systems is less about isolated model accuracy and more about designing robust, observable, and cost-aware pipelines that integrate multiple components seamlessly.

6.2 Connection to Course Material

Prompting & LLM Safety. The `prompts.py` module defines structured prompts (XML-style sections: `<role>`, `<task>`, `<response_requirements>`, `<forbidden_claims>`, `<output_contract>`) for Bedrock. Hierarchical prompting places constraints in the system field to override user input.

Model Training & Selection. ArtGuard applies transfer learning by fine-tuning a Swin Transformer pretrained on ImageNet (variant iii). Techniques include ReduceLROnPlateau scheduling, early stopping (patience=10), weighted BCE loss (10× inauthentic), pinned memory, and augmentation. Model choice (Swin-Tiny, 28M vs. Swin-Base, 88M) follows scaling laws, matching capacity to dataset size to reduce overfitting.

Model Serving & Infrastructure. Modal serverless GPUs handle inference (on-demand spin-up, no persistent server), suited to latency-tolerant, low-throughput, bursty workloads. The architecture separates CPU (ECS Fargate) and GPU (Modal) workloads, alongside Bedrock, DynamoDB, S3, OpenSearch Serverless, CloudFront, and Terraform, reflecting a “mixture of compute” design.

Data & Storage Systems. A lightweight feature store uses DynamoDB (metadata) and S3 (raw/processed images). The ETL pipeline ingests MET and Wikidata, preprocesses images into patches, and follows a medallion flow (Bronze raw, Silver processed, Gold labeled). S3 lifecycle tiering (Standard → IA → Glacier) reduces storage cost, with deletion policies for user uploads.

RAG System. Hybrid search (`kb_search_type="HYBRID"`) combines BM25 keyword search with dense retrieval (Titan Embed Text v2) in OpenSearch Serverless, enabling both exact matching and semantic similarity.

Evaluation. Evaluation includes training/validation metrics (loss, accuracy, bits-per-byte) tracked via W&B and a large test suite (626 tests). We use Langfuse for fine tuning our RAG system prompt on the following metrics: faithfulness, alignment, patch reference and hedging.

7 Sources

References

- [1] N. Larson, *Fine arts experts institute: Lab sleuths in Geneva help art world uncover fakes*, ArtDaily, 2014. [Online]. Available: https://artdaily.com/index.asp?int_sec=11&int_new=73562
- [2] E. Azimi, A. Ashtari, and J. Ahn, “Patch-based oil painting forgery detection based on brushstroke analysis using generative adversarial networks and depth visualization,” *Applied Sciences*, vol. 15, no. 1, p. 75, 2025. <https://doi.org/10.3390/app15010075>
- [3] L. Schaerf, C. Popovici, and E. Postma, “Art authentication with Vision Transformers,” *arXiv preprint arXiv:2307.03039*, 2023. <https://doi.org/10.48550/arXiv.2307.03039>
- [4] M. Zanardelli, F. Guerrini, R. Leonardi, and N. Adami, “Image forgery detection: A survey of recent deep-learning approaches,” *Multimedia Tools and Applications*, vol. 82, pp. 17521–17566, 2023. <https://doi.org/10.1007/s11042-022-13797-w>
- [5] T. Dobbs and Z. Ras, “On art authentication and the Rijksmuseum challenge: A residual neural network approach,” *Expert Systems with Applications*, vol. 200, p. 116933, 2022. <https://doi.org/10.1016/j.eswa.2022.116933>
- [6] Z. Huang et al., “So-Fake: Benchmarking and explaining social media image forgery detection,” *arXiv preprint arXiv:2505.18660*, 2025. <https://doi.org/10.48550/arXiv.2505.18660>
- [7] Art Recognition AG, *Art authentication using artificial intelligence*, 2023. [Online]. Available: <https://art-recognition.com>
- [8] Hephaestus Analytical, *Pictology: AI-based art authentication technology*, 2023. [Online]. Available: <https://hephaestus-analytical.com>
- [9] Norval AI, *AI-powered authentication of Norval Morrisseau artworks*, 2022. [Online]. Available: <https://norvalai.com>
- [10] Wikimedia Commons, *The free media repository*. [Online]. Available: https://commons.wikimedia.org/wiki/Main_Page
- [11] The Metropolitan Museum of Art, *The Metropolitan Museum of Art Open Access dataset*. [Online]. Available: <https://github.com/metmuseum/openaccess>
- [12] Anthropic, *Claude prompting best practices*. [Online]. Available: <https://platform.claude.com/docs/en/build-with-claude/prompt-engineering/claude-prompting-best-practices>
- [13] Boccuzzo et al., “Art Forgery Detection using Kolmogorov Arnold and Convolutional Neural Networks.”
- [14] T. Dobbs et al., “Contemporary Art Authentication with Large-Scale Classification.” <https://doi.org/10.3390/bdcc7040162>